

JUVION PARA MICRO SAAS

Preparação, infraestrutura e mapeamento de componentes para migração de script local para API na nuvem.

Status:	Planejamento Inicial
Autor:	Antigravity AI (Pair Programming)
Data:	Junho de 2026
Repositório de Backup:	github.com/wildfire96/juvion

1. Contexto do Sistema

Atualmente, o projeto **Juvion** opera como uma suite de scripts locais em Python com o objetivo de prospectar potenciais clientes (leads), analisar a presença digital deles (tecnologias, performance) e propor soluções de design/SEO via análise automatizada por inteligência artificial (Gemini API).

Mapeamento do Código Atual:

- **main.py**: Orquestrador do fluxo (coleta os leads, tira screenshots e compila em JSON).
- **scraper.py**: Automatiza buscas no Google Maps utilizando a biblioteca Playwright.
- **analyzer.py**: Verifica o stack tecnológico do site e chama a API do Google PageSpeed.
- **vision_ai.py**: Envia as capturas de tela dos sites para análise visual usando a API do Gemini.
- **gerar_mensagens.py**: Estrutura a mensagem personalizada para abordagem comercial.

2. Desafios da Migração para Micro SaaS

Migrar um script de CLI local para um modelo multi-usuário (SaaS) envolve resolver alguns gargalos fundamentais de performance, concorrência e custos:

Tempo de Resposta: O processo completo (Maps + Playwright + Gemini) pode demorar mais de 1 minuto por lead. O HTTP síncrono clássico resultaria em timeout no navegador.

Consumo de RAM: Executar instâncias do Chromium em servidores na nuvem consome muita memória. Soluções como Docker otimizado ou APIs como Browserless.io reduzem custos.

Segurança: Chaves de API sensíveis (Gemini, PageSpeed) devem ficar estritamente no backend, protegidas de vazamentos ou acessos não autorizados.

3. Arquitetura Alvo (Micro SaaS)

Para transformar esse fluxo em um SaaS produtivo e escalável, propõe-se a seguinte divisão de responsabilidades:

Componente	Tecnologia Proposta	Função no SaaS
Front-end	React / Next.js / Tailwind	Interface de usuário, dashboard, status da prospecção.
Back-end API	FastAPI (Python)	Recebe requisições do front-end e gerencia tarefas assíncronas.
Fila de Tarefas	Celery + Redis	Executa o scraper, screenshots e análise do Gemini em background.
Banco de Dados	Supabase (PostgreSQL)	Persistência dos leads, logs de auditoria e cadastro de usuários.
Armazenamento	Supabase Storage Buckets	Hospeda as capturas de tela dos leads para exibição na UI.

4. Plano de Ação para Implementação

Fase 1: Refatoração das Funções Core (Próximo Passo)

Isolar a lógica atual de prospecção e análise de site em funções puras (sem salvar em arquivos .json locais), preparadas para receber parâmetros e salvar no banco.

Fase 2: Estrutura do Backend e Banco

Configurar a API FastAPI com endpoints e as tabelas de banco de dados para armazenar leads e status do processo (Ex: 'pendente', 'processando', 'concluido').

Fase 3: Processamento Assíncrono (Workers)

Acoplar o Playwright e a chamada do Gemini a um worker assíncrono para garantir que a API responda instantaneamente.

Fase 4: Painel Web (Front-end)

Construir uma UI limpa e de alta fidelidade visual, permitindo aos usuários configurar filtros de busca e visualizar os relatórios gerados.